Static analysis with focus on using language models and Hoare logic

Rashid Harvey
Freie Universität Berlin
IoT & Security Seminar Report

***Abstract*—Abstract**
***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

A. IoT Security

The Internet of Things (IoT) has grown into one of the largest computing domains in existence. Industry trackers estimate that the number of connected IoT devices reached roughly 21 billion worldwide in 2025, growing at about 14% per year and projected to approach 39 billion by 2030, [1] as the worldwide IoT market continues its rapid expansion. [2] This ubiquity has not gone unnoticed by attackers: already in 2016, Gartner predicted that "by 2020, more than 25 percent of identified attacks in enterprises will involve IoT, although IoT will account for less than 10 percent of IT security budgets." [3]

Indeed, security is frequently an afterthought in IoT development. Profit-driven business models and pressure to minimize time-to-market lead manufacturers to overlook security considerations and to ship devices that are vulnerable by design. [4]

Compounding this, IoT devices commonly run custom-built, vendor-specific firmware rather than the standardized, widely-audited software stacks found in general-purpose computing. The resulting heterogeneity of standards and communication stacks means that traditional security countermeasures cannot be directly applied to IoT technologies, [5] and the inherent complexity of so many heterogeneous entities interacting greatly complicates the design and deployment of interoperable security mechanisms. [6] Bespoke firmware is also more prone to weak programming practices and unaudited code paths than mature, mainstream software. [4]

A further obstacle is that IoT endpoints are typically severely resource-constrained. The IETF characterizes such "constrained nodes" by hard limits on memory (RAM), code size (ROM/Flash), available computation, and power, with the smallest device class offering well under 10 KiB of RAM. [7] Generally, "limited storage, power, and computational capabilities hinder addressing IoT security requirements." [4]

Finally, even once vulnerabilities are discovered, IoT devices are notoriously difficult to update. Most devices lack a built-in secure firmware update mechanism, so that "critical security vulnerabilities cannot be fixed, and the IoT devices can become a permanent liability." [8] Many are deployed with little or no infrastructure for applying patches, [9] and even where update mechanisms exist they are routinely neglected: a large share of devices run outdated firmware, with surveys reporting that roughly 40% of users never update at all. [4]

B. Static Analysis

Static analysis is the analysis of code, binaries or other artifacts to check certain properties without, in contrast to dynamic analysis, executing any code. [10]

Static analysis examines code *without* executing it; dynamic analysis examines it *by* executing it.

- **Examples fo static analysis:** linting, formatting, syntax checking, type checking, flow analysis.
- **Examples for dynamic analysis:** testing, emulation.

Static analysis is efficient, i.e. it runs quickly and cheaply, and catches errors early, before runtime. Its limitations are theoretical (the halting problem) and a potentially high false-positive rate.

C. Large Language Models for Coding

LLM have seen massive advancements for coding and security. For example, looking at the "SWE-bench Verified" benchmark of 500 human-selected GitHub Issues the best-performing model, Claude 2, was only able to solve 1,96% of the issues at the time of the benchmarks inception in 2023. [11]. This stands in stark contrast to the current state-of-the-art: Claude Fable 5 solves 95% of the same benchmark at the time of the models release. [12]

Even if one was to account for the limited effectiveness of public benchmarks, this shows quite clearly that LLMs can effectively be used for developing code. [1]

The US government also seams to agree. On the 12th of June 2026 they forced Anthropic to take down Fable 5, the already down-graded safe-guarded version of Mythos 5, from public access after just about 76 hours by imposing a national-security export control. [15]

Earlier, AISLE had already found several major security issues in famously hard-audited, mature codebases like OpenSSL. [16]

[1] For example, the benchmarks might be leaky, [13], and the models train and memorize the solutions instead of learning the general principles. [14]

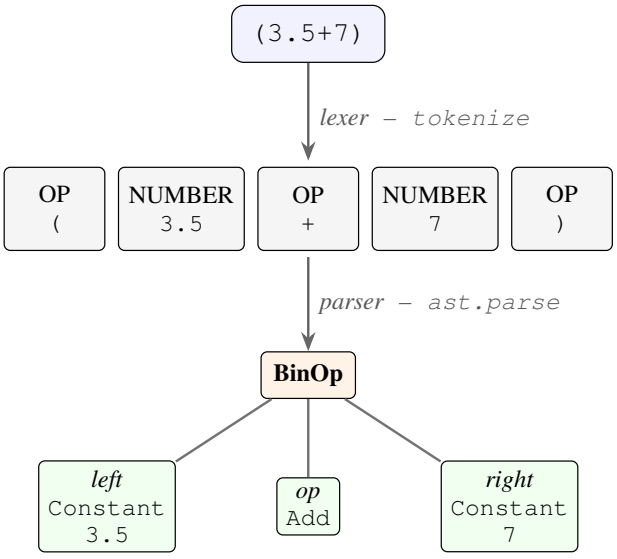

Fig. 1. Lexing and parsing of the expression `(3.5+7)` in Python

II. Static Analysis Methods

A. Lexical Methods and Methods Using the Abstract Syntax Tree Representation

Lexical methods and methods using the abstract syntax tree (AST) representation are pretty ”simple” methods, you would often summarize as linting.

Lexical analysis simple means breaking down text into the smallest semantic units ”tokens” and analyzing them. The AST is a representation of the code as a tree, which allows for more depth. The AST is what a compiler, transpiler, interpreter and most linters and formatters will consume.

Examples

1) Syntax errors
2) Name errors or misspellings
3) Missing definitions and declarations
4) Rule matching, e.g. leaked secrets

B. Type checking

The idea behind types is that any data on a digital computer is simply some bitstring. The interpretation of this bitstring is what makes the data a value.

The data only makes sense if we define operations on it, but these operations depend on the type. For example, in Python the + operator for integers does integer addition, but for strings it does string concatenation. However, if you try to ”add” an integer with a string, you get a type error. In Python type checking is done at runtime and is therefore called ”dynamic type checking”. However, for static analysis, we can check the types ahead of run-time ”static type checking”. [17], [18]

In his paper “A Theory of Type Polymorphism in Programming“ Robin Milner argues that well-typed programs cannot “go wrong“ and that therefore programmers should use a formal type discipline. [19]

C. Flow Control

A further class of semantic methods reasons about how information moves through a program. *Information-flow analysis* tracks how the values of variables depend on one another, with the goal of certifying that secret data never reaches a public output. [20], [21] A flow from a variable x to a variable y, written $x \to y$, means that observing y may reveal something about x. Two kinds of such flows are distinguished.

Explicit flows. The most direct case is an assignment that copies a value. In the program $y := x$ all information from x flows to y. The flow need not be total: in $y := x/8$ not all, but some information may flow from x to y, since y no longer determines x exactly. Both are *explicit* flows, denoted $x \to y$, because the dependency is carried directly by the data passed in the assignment.

Implicit flows. Information can also leak through the control flow itself, without any direct assignment from the source. Consider:

$y := 0$
if $x \geq 8$ **then**
  $y := 1$
end if

Here x is never assigned to y, yet some information still flows from x to y: after the final line, observing y tells us whether $x \geq 8$. This is an *implicit* flow, again denoted $x \to y$. In general, any conditional control-flow change encodes information about the values that guided it, so a sound analysis must account for implicit as well as explicit flows. [20], [21] Practical enforcement tools build on this distinction; the Jif/JFlow extension of Java, for example, lets programmers annotate data with security labels that the compiler then checks against both flow kinds. [22]

Related flow-analysis techniques include control-flow analysis, which examines the possible paths of execution, and taint analysis, which follows untrusted (“tainted”) inputs to ensure they cannot reach sensitive operations unsanitized.

D. Case study: Rust

Rust is a low-level programming language started in 2006 by Graydon Hoare as a side project, then officially sponsored by Mozilla in 2009 and it finally reached maturity (v1.0) in 2015. Rust was specifically created to solve the flaws of C/C++. [23], [24] This section tries to summarize how this is done in Rust.

“Who doesn’t want memory safety, *and* fast performance, *and* a friendly compiler, *and* great tooling, among a host of other wonderful features¿‘ [25]

The Rust language has a very strict compiler which includes not just type checking but also a borrow checker and more. [25] These checks lead to the conclusion that regular Rust code compiled through this strict compiler is also ”safe”: “If it compiles, it works“ is a common community lore. This reminds onself of Milners idea, that says well-types programs cannot go wrong. [19]

"Rust's type system and runtime guarantee the absence of data races, buffer overflows, stack overflows, and accesses to uninitialized or deallocated memory." [26]

The correctness of a small subset of Rust was also proven in 2018 in the paper "RustBelt: Securing the Foundations of Rust" where the authors do a formal, machine-checked safety proof. [27]

1) Case studies: Large-scale industry data supports this: Microsoft reports that roughly 70% of its annually assigned CVEs are memory-safety issues, [28], [29], while Android saw memory-safety vulnerabilities fall from 76% to 24% after shifting new development to memory-safe languages. [30]

III. Formal Methods

A. Introduction to Formal Methods

classical static analysis finds symptoms; formal methods prove properties

When applying formal methods first a formal model is created. The kind of model used depends on the use case After the model is set up, model checking must be applied, this is called formal verification.

The beauty of Hoare logic

B. Overview of Methods

1) Model-based formal specification languages like Z, Vienna Development Model (VDM), and B
2) Interactive proof assistants like Rocq (formerly Coq), [31] Isabelle/HOL [32] or Lean [33]
3) Verification-aware programming languages like Dafny, [34] SPARK, F*, [35] Why3, Viper, Whiley or hax

C. Examples

1) Dafny: In the original paper on Dafny, Leino gave the challenges of directly using interactive proof assistants and satisfiability-modulo-theories (SMT) solvers as main reason for the creation of Daphny. The idea is that the programmer doesn't have to interact with any proof assistant or solver directly, only make certain annotations [34]. Dafny compiles down to C#, Java, JavaScript, Go, and Python

D. Limitations

1. Gaps: Scalability, Real-Time Performance, Cooperation between Formal Methods, Adaptability, Lack of Focus on Security & Privacy ?, Usability,

IV. Discussion

-¿ build "theoretical emulators"

V. Conclusion

References

[1] S. Sinha, "State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally," https://iot-analytics.com/number-connected-iot-devices/, Oct. 2025, connected-device count: 21.1 billion by end of 2025 (14% YoY); forecast 39 billion by 2030 (CAGR 13.2%).

[2] Statista, "Internet of Things — worldwide (market outlook)," https://www.statista.com/outlook/tmo/internet-of-things/worldwide, 2025, statista IoT market-outlook dashboard; worldwide IoT market revenue projections. User-supplied source for market growth.

[3] Gartner, "Gartner says worldwide IoT security spending to reach $348 million in 2016," https://www.gartner.com/en/newsroom/press-releases/2016-04-25-gartner-says-worldwide-iot-security-spending-to-reach-348-million-in-2016, Apr. 2016, press release, 25 Apr 2016. Verbatim prediction: "By 2020, more than 25 percent of identified attacks in enterprises will involve IoT, although IoT will account for less than 10 percent of IT security budgets".

[4] N. Neshenko, E. Bou-Harb, J. Crichigno, G. Kaddoum, and N. Ghani, "Demystifying IoT security: An exhaustive survey on IoT vulnerabilities and a first empirical look on internet-scale IoT exploitations," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2702–2733, 2019, multi-claim intro source: "time-to-market ... overlook security considerations"; "limited storage, power, and computational capabilities hinder addressing IoT security requirements"; improper patch/software-update capabilities; 40% of consumers never update firmware; weak programming practices in vendor firmware.

[5] S. Sicari, A. Rizzardi, L. A. Grieco, and A. Coen-Porisini, "Security, privacy and trust in Internet of Things: The road ahead," *Computer Networks*, vol. 76, pp. 146–164, 2015, "Traditional security countermeasures cannot be directly applied to IoT technologies due to the different standards and communication stacks involved".

[6] R. Roman, J. Zhou, and J. Lopez, "On the features and challenges of security and privacy in distributed Internet of Things," *Computer Networks*, vol. 57, no. 10, pp. 2266–2279, 2013, heterogeneity of IoT entities complicates interoperable security; need for efficient cryptographic algorithms even on 8-bit/16-bit devices.

[7] C. Bormann, M. Ersue, and A. Keränen, "Terminology for constrained-node networks," RFC 7228, Internet Engineering Task Force (IETF), May 2014, canonical definition of constrained nodes: limits on code size (ROM/Flash), state/buffers (RAM), computation, and power; device Classes 0/1/2 (Class 0: ¡ 10 KiB RAM, ¡ 100 KiB Flash).

[8] K. Zandberg, K. Schleiser, F. Acosta, H. Tschofenig, and E. Baccelli, "Secure firmware updates for constrained IoT devices using open standards: A reality check," *IEEE Access*, vol. 7, pp. 71 907–71 920, 2019, "most IoT devices lack a built-in secure firmware update mechanism. Without such a mechanism ... critical security vulnerabilities cannot be fixed, and the IoT devices can become a permanent liability"; targets devices with ¡ 32 kB RAM / 128 kB flash.

[9] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, "Understanding the Mirai botnet," in *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, 2017, pp. 1093–1110, "rampant use of insecure default passwords in IoT products"; "the absence of security best practices ... has created an IoT substrate ripe for exploitation"; infected devices had "little infrastructure to effectively apply security patches".

[10] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '77)*. ACM, 1977, pp. 238–252.

[11] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024, baseline: best model (Claude 2) solves 1.96% of the issues.

[12] Anthropic, "Claude fable 5 and claude mythos 5 system card," https://www-cdn.anthropic.com/d00db56fa754a1b115b6dd7cb2e3c342ee809620.pdf, 2026, released 9 June 2026. SWE-bench Verified: Fable 5 95.0%, Mythos 5 95.5%, Opus 4.8 88.6%.

[13] R. Aleithan, H. Xue, M. M. Mohajer, E. Nnorom, G. Uddin, and S. Wang, "SWE-Bench+: Enhanced coding benchmark for LLMs," *arXiv preprint arXiv:2410.06992*, 2024, audit of SWE-bench: 32.67% of resolved instances had solution leakage, 31.08% passed via weak tests; filtering dropped SWE-Agent+GPT-4 from 12.47% to 3.97%. Same issues in Lite/Verified.

[14] S. Liang, S. Garg, and R. Zilouchian Moghaddam, "The SWE-bench illusion: When state-of-the-art LLMs remember instead of reason," *arXiv preprint arXiv:2506.12286*, 2025, memorization/contamination: models identify buggy file paths up to 76% from issue text alone vs. 53% for out-of-benchmark repos.

[15] Anthropic, "Statement on the US government directive to suspend access to fable 5 and mythos 5," https://www.anthropic.com/news/fable-mythos-access, Jun. 2026, export-control directive received 12 June 2026, 5:21pm ET; Fable 5/Mythos 5 launched 9 June 2026 (roughly 72 h earlier).

[16] AISLE, "AISLE discovered 12 out of 12 OpenSSL vulnerabilities," https://aisle.com/blog/aisle-discovered-12-out-of-12-openssl-vulnerabilities, Jan. 2026, aI-driven discovery of 12 CVEs (incl. CVE-2025-15467, HIGH, CVSS 9.8) in a mature, heavily audited codebase; some bugs date back to 1998. AISLE's system was powered by Claude Opus 4.6.

[17] B. C. Pierce, *Types and Programming Languages*. Cambridge, MA: MIT Press, 2002, standard reference. Defines a type system as a static method; contrasts static type checking with dynamic typing.

[18] Python Software Foundation, "The python language reference: Data model," https://docs.python.org/3/reference/datamodel.html, 2026, official source for runtime/dynamic typing: every object has a type; operators desugar to type-dependent special methods, e.g. `a+b` calls `type(a).__add__` / `type(b).__radd__`.

[19] R. Milner, "A theory of type polymorphism in programming," *Journal of Computer and System Sciences*, vol. 17, no. 3, pp. 348–375, 1978, maxim "well-typed programs cannot go wrong" — academic root of the slogan "if it compiles, it works".

[20] D. E. Denning and P. J. Denning, "Certification of programs for secure information flow," *Communications of the ACM*, vol. 20, no. 7, pp. 504–513, 1977, classic primary source on information-flow analysis.

[21] A. Sabelfeld and A. C. Myers, "Language-based information-flow security," *IEEE Journal on Selected Areas in Communications*, vol. 21, no. 1, pp. 5–19, 2003, standard survey.

[22] A. C. Myers, "JFlow: Practical mostly-static information flow control," in *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '99)*. ACM, 1999, pp. 228–241, the concrete information-flow tool (implemented as Jif); counterpart to the Denning and Sabelfeld/Myers theory references.

[23] C. Thompson, "How Rust went from a side project to the world's most-loved programming language," https://www.technologyreview.com/2023/02/14/1067869/rust-worlds-fastest-growing-programming-language/, Feb. 2023, single source covering the full origin sentence: 2006 side project by Hoare, Mozilla official sponsorship 2009, v1.0 in 2015, motivated by C/C++ memory-safety flaws.

[24] G. Hoare, "10 years of stable rust: An infrastructure story," https://rustfoundation.org/media/10-years-of-stable-rust-an-infrastructure-story/, May 2025, guest post by Rust's initial author. Primary source for the origin story: project begun 2006, "near-decade of development" before the 1.0 release (15 May 2015). Mozilla sponsorship from 2009.

[25] S. Klabnik and C. Nichols, *The Rust Programming Language*, 3rd ed. No Starch Press, 2025, official book; chapters on ownership and borrowing. Freely available: https://doc.rust-lang.org/book/.

[26] N. D. Matsakis and F. S. Klock II, "The Rust language," in *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology (HILT '14)*, ser. Ada Letters, vol. 34, no. 3. ACM, 2014, pp. 103–104, source of the quote on Rust's guarantees (absence of data races, buffer/stack overflows, use of uninitialized/deallocated memory).

[27] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, "RustBelt: Securing the foundations of the Rust programming language," *Proceedings of the ACM on Programming Languages*, vol. 2, no. POPL, pp. 1–34, 2018, formal safety proof for a core subset of Rust incl. well-encapsulated unsafe code.

[28] M. Miller, "Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape," Presentation at BlueHat IL 2019; slides at https://github.com/microsoft/MSRC-Security-Research, Microsoft Security Response Center, Feb. 2019, primary source for the roughly 70% figure: roughly 70% of the CVEs Microsoft assigns each year are memory-safety issues, stable for 12+ years.

[29] Microsoft Security Response Center, "We need a safer systems programming language," https://msrc.microsoft.com/blog/2019/07/we-need-a-safer-systems-programming-language/, Jul. 2019, mSRC explicitly argues for memory-safe languages such as Rust; roughly 70% of CVEs are memory-safety issues despite code review, training, and static analysis.

[30] J. Vander Stoep and A. Rebert, "Eliminating memory safety vulnerabilities at the source," https://security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html, Sep. 2024, case study: Android memory-safety vulnerabilities fell from 76% (2019) to 24% (2024) over 6 years as new development shifted to memory-safe languages, well below the roughly 70% industry norm.

[31] The Rocq Development Team, "The rocq prover (formerly coq), version 9.0," https://rocq-prover.org/, 2025, renaming Coq to The Rocq Prover; first release under the new name (v9.0, 12 March 2025).

[32] T. Nipkow, L. C. Paulson, and M. Wenzel, *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, ser. Lecture Notes in Computer Science. Springer, 2002, vol. 2283.

[33] L. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer, "The Lean theorem prover (system description)," in *Automated Deduction — CADE-25*, ser. Lecture Notes in Computer Science, vol. 9195. Springer, 2015, pp. 378–388.

[34] K. R. M. Leino, "Dafny: An automatic program verifier for functional correctness," in *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR-16)*, ser. Lecture Notes in Computer Science, E. M. Clarke and A. Voronkov, Eds., vol. 6355. Springer, 2010, pp. 348–370.

[35] N. Swamy, C. Hriţcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoué, and S. Zanella-Béguelin, "Dependent types and multi-monadic effects in F*," *ACM SIGPLAN Notices (POPL 2016)*, vol. 51, no. 1, pp. 256–270, 2016.